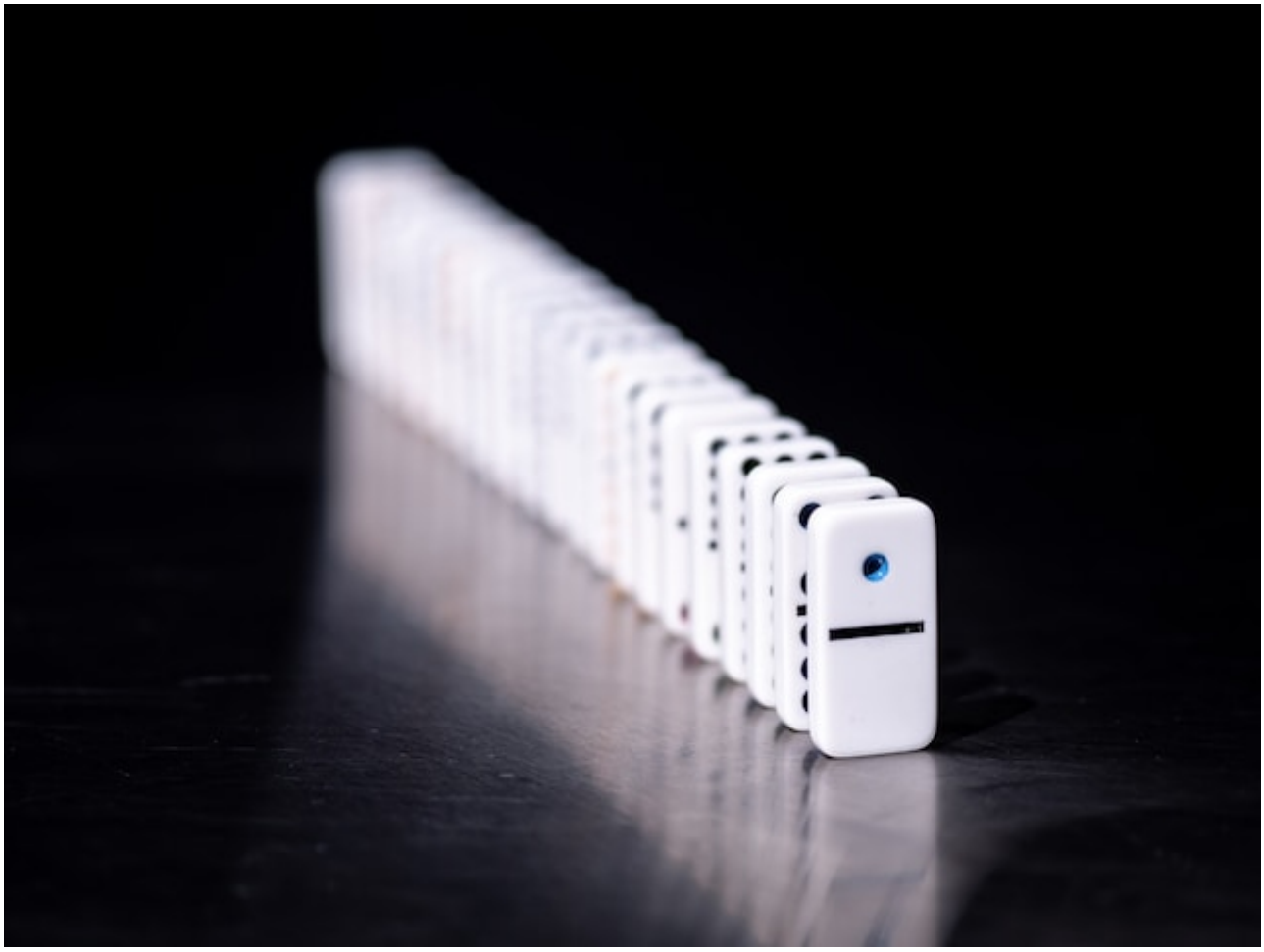
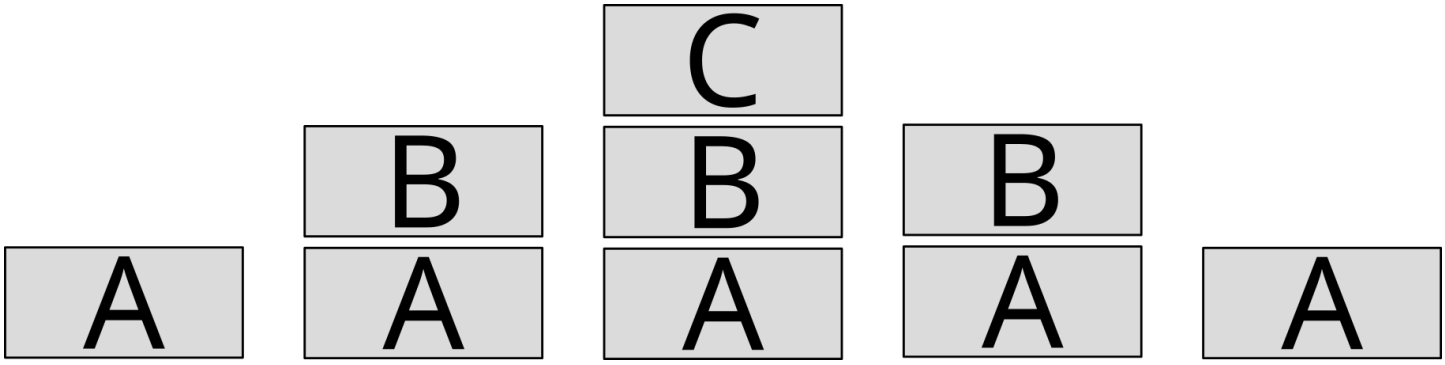


Chapter 4 - Stacks and queues

Even if programming in general usually is about abstract concepts coders are often asked to write programs that solve some kind of problem in real life. That means that many times programs have to replicate somehow things we see in real life. And that affects also data structures. In this chapter we are going to see two related data structures that are better explained if we can picture a real life example.

Stacks

Let's start with **stacks**. Stacks are like a pile of plates. You normally add plates to the top of the pile and when you need a plate, you take the first one at the top. This kind of interaction is called LIFO, that comes from "Last In first Out". The plates go out in opposite order they came in. Given the plates on a stack, the last that went in is the next one in line to come out.



Like in the image. First we add "A" to the stack. Then we add "B" and it goes on top of "A". And we also add "C" to the stack and it goes on top of "B". We can't get "A" or "B" before we take out "C". And same with "A" and "B". We have to take out "B" to get to "A".

But this data structure is not meant to be used in cases where we need "random" access to the elements. There are plenty of cases where we want just that, get the elements in the inverse order that they were stored.

For example, programming languages themself use stacks to store data relative to the scope we are executing when we call a function. So functions can call other functions and at the same time, information relative to each scope starts to pile up in the stack. And this way is very easy to find the way back to the first scope when the functions start to return: program starts to get out the information from the stack.

Ok. You probably have now a good understanding of what a stack is and can even start to think on ways to implement one. Let's think on it for a moment. A stack can be implemented with an array, where program creates an array of a certain size and then it starts to fill it up. This is quite simple, but the problem with this approach is the fixed lenght nature of arrays. The stack will be limited to the size of the array before incurring in the expense of changing to a bigger array or alternatively the array size is a hard limit for the stack size.

Another way of implement a stack is use it like a singly linked list, where instead of a head we have a **top**. The top element needs to point to next element in the stack. And that element points to the next one, and so on.

The actions we want to perform on a stack are very limited. We want to add an element to the top (i.e. to insert at the front). And we want to take out the top element (i.e. remove from the front). This two actions are called **push** and **pop** respectively.

As we saw in the previous chapter, inserting and removing elements from the front of the list was of time complexity $O(1)$. So stacks implemented this way are very efficient and doesn't have the length problem as with arrays.

Stacks can be implemented with either of those solutions, but we are going to focus on the one based on a singly linked list. We didn't implement a push or pop from the front of the list functions in the previous chapter, but if we had done it, we could use directly those. Instead we are going to implement this structure from scratch, as it is very limited and simple.

First, the node to store the data should be very familiar:

```
class Node:
    def __init__(self, data=None, next=None):
        self.data = data
        self.next = next

    def __repr__(self):
        return f'<Node: {self.data}>'
```

We don't even need the "prev" pointer as it is not going to be used in this structure. Next we can start our Stack class:

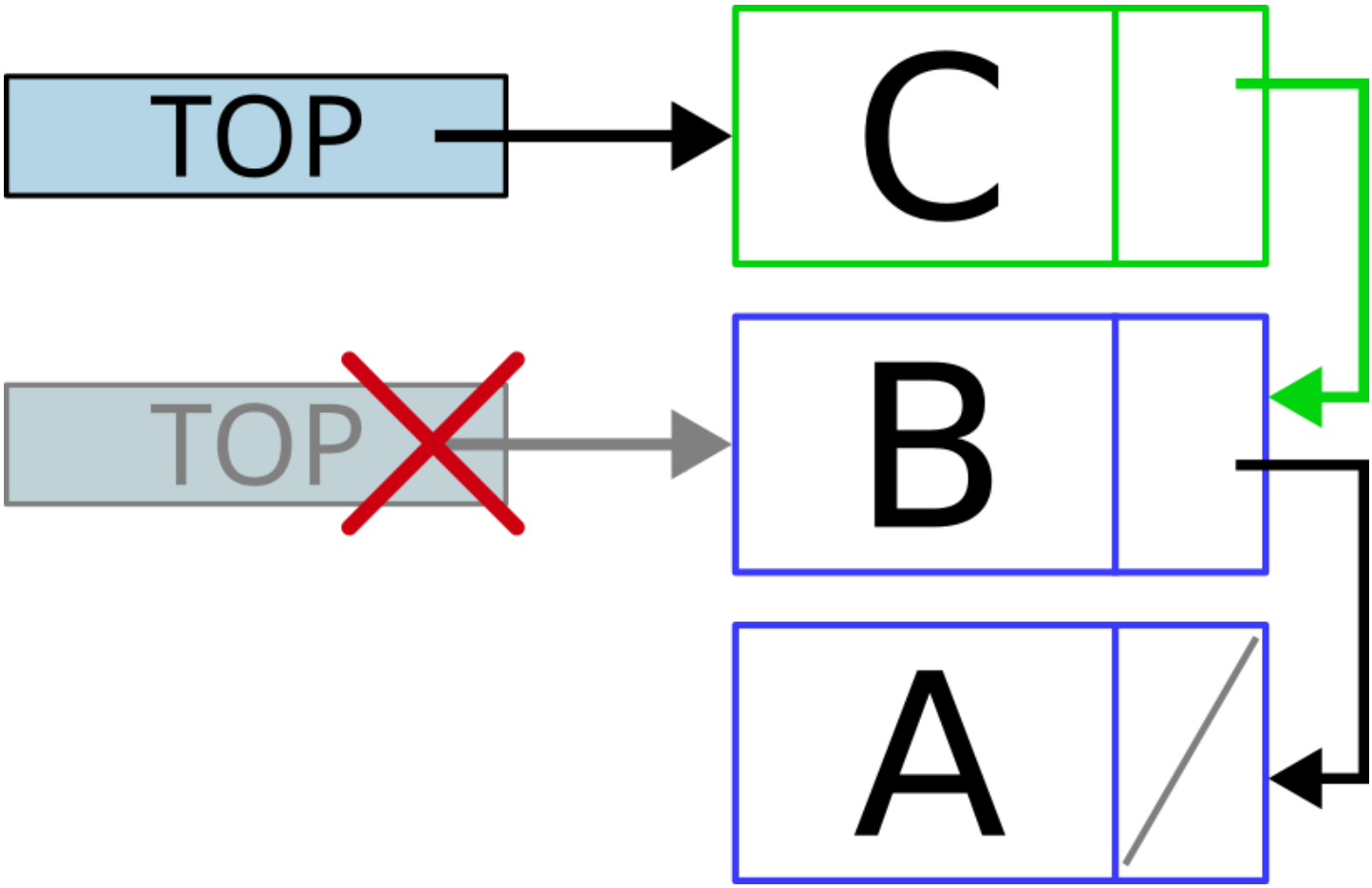
```
class Stack:
    def __init__(self):
        self._top = None
        self._size = 0

    def __len__(self):
        return self._size

    def peek(self):
        """
        Returns the value of the top node without altering the stack
        """
        return self.top.data if self.top else None

    def push(self, data):
        """
        Add an element to the stack

        Parameters:
        - 'data': Data/value being added

        Returns: None
        """
        return
```

```
    def pop(self):
        """
        Remove the top node from the stack and return its content

        Parameters: None

        Returns: The content of the node or None if stack is empty
        """
        return
```

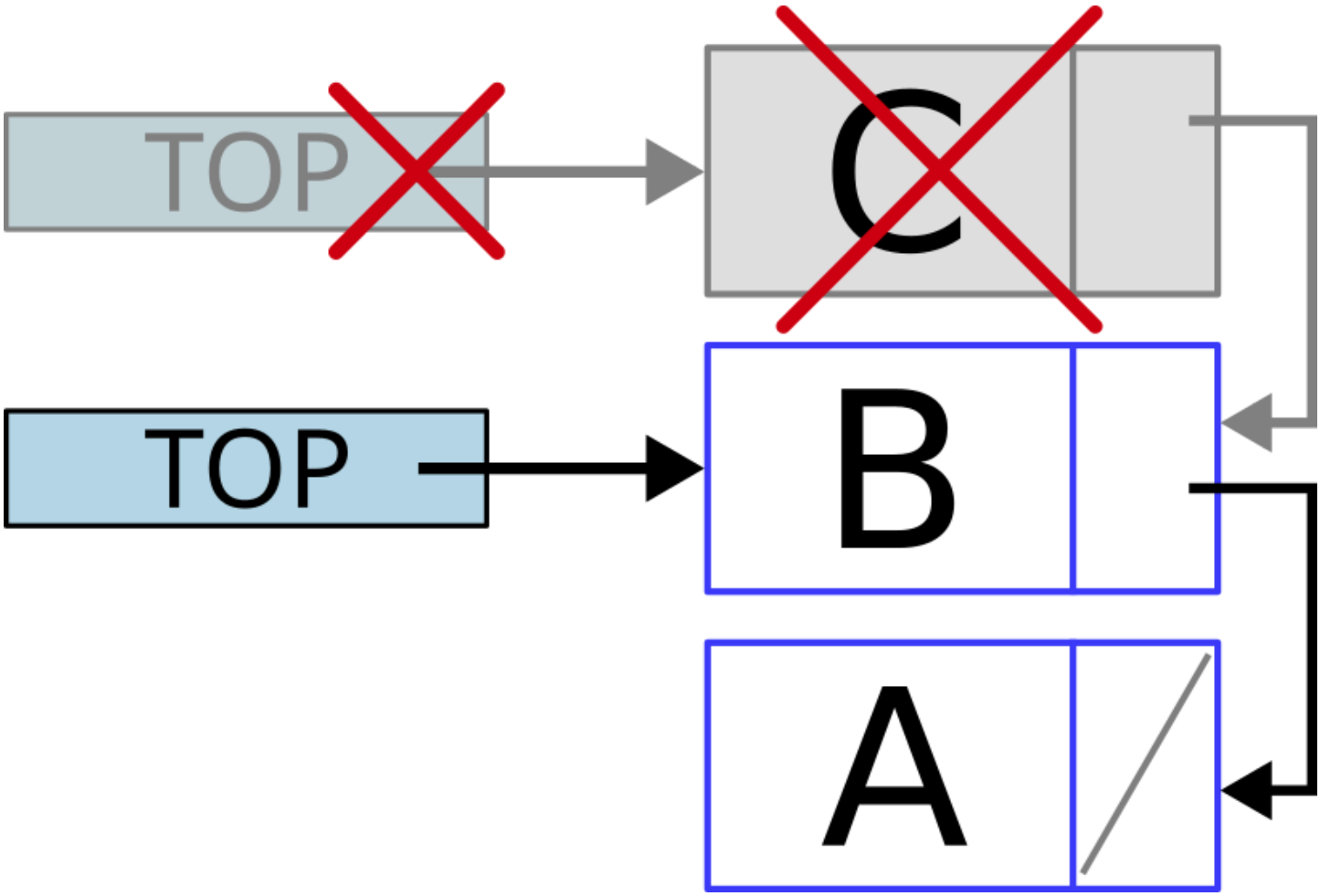
So, instead of a head, we have a "top" pointer. A "size" property has been added, but it's not mandatory. And another nice feature has also been added, that is not mandatory either, but sometimes is used with stacks, and it is the **peek** action. Peek is used to take a look to the top of the stack. So it is very easy to implement.

That leaves us with the `push` and `pop` actions/functions. And now it's your turn. Open the Moodle environment and complete the exercise number 1 of chapter 4 where you will implement these methods.

The `push` function should be very easy. It just creates a new node that points to where the top pointer is pointing and then updates top pointer to point to the new node.



The `pop` function should also be very easy. Update the top pointer to where the top node was pointing and delete the old top element. The function should return the value of the node being removed. If the stack was empty just return None.



You will also have to implement a `__repr__()` method that returns the content of the stack like this:

```
<Stack (3 elements): ['C', 'B', 'A']>
```

The list is in order from top to bottom and should be empty if the stack is empty.

Notice

Don't continue until you finish the exercise.

And now let's take into practice this stucture. You are going to complete the exercise number 2 of the chapter 4 of the Moodle environment, where you have to write a function that checks whether a string containing brackets symbols (`()[]{}`) is balanced or not using a stack.

The idea is simple, when you encounter an opening bracket symbol (`(` or a `{` or a `[`) you will push it to the stack. When you encounter a closing bracket symbol (`)` or a `}` or a `]`), pop the one on the top of the stack and check if they match. If they don't match, you return an error. If you get to the end of text without errors, return some other text.

Notice

Don't continue until you finish the exercise.

Queues

The other data structure that somehow mimics real life is the **Queue**. In this case you can picture whatever queue or line you have to do any normal day: in the metro, in the shop, in your favourite coffee shop, ...

This kind of structures work in a **FIFO** way. FIFO comes from "First In, First Out". So in any kind of line, if everybody behaves nicely (as they should), they will wait for the persons that came before to be served before expecting any service. The elements of a queue go out in the same order they came in.

The two actions that need to be performed with queues are **enqueue** and **dequeue**. There can be more, but these two are mandatory.

Queues are not difficult to picture and it's quite easy to understand how this structure works. But how is implemented is another thing. Same as with the stack, queues can be implemented with an array. But in this case it's quite inefficient. As you need to operate in both ends of the array, you end up either inserting at the front or deleting from the front. And both actions are time consuming with arrays. They are of time complexity $O(n)$ as the whole array has to be shifted forward or backwards.

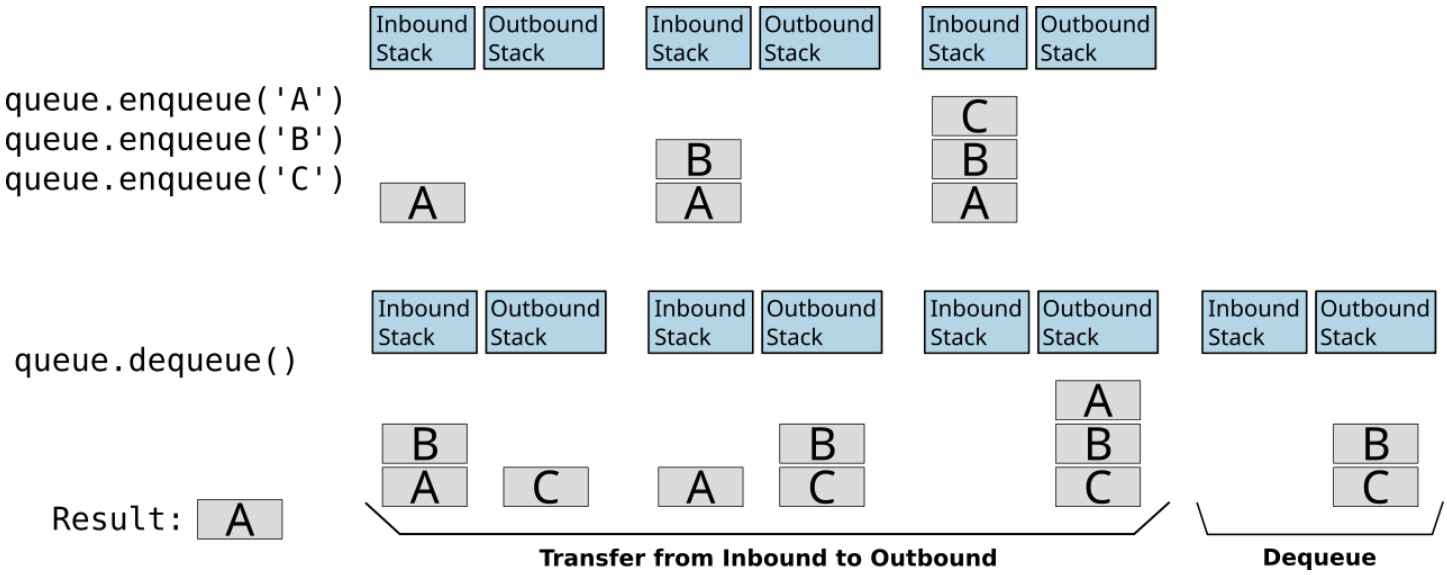
Stack based queue

One solution to this is the "two stack based queue", where two stacks are used. Stacks themself can be implemented with arrays or list structures.

In this kind of solution there is an Inbound stack and an Outbound stack. When an `enqueue` operation is performed with some data, it is added to the end of the Inbound stack. And when the `dequeue` operation is requested, a value is retrieved from the end of the Outbound stack. This kind of operations with arrays are of time complexity $O(1)$ if there is still room in the array.

But these are two different stacks, how are they connected? When a `dequeue` operation is requested and if the Outbound stack is empty, the content of the Inbound stack is transfered to the Outbound stack, one element pop and push at a time. Pop from Inbound stack and push to Outbound stack... until all elements are transfered. this way, when we pop from Outbound, we will get the element that has been waiting the longest. As we have seen, push and pop are of time complexity $O(1)$ regardless of how is the the stack implemented, with arrays or lists. Of course, if implemented with arrays there is the same problem with the size of the arrays as explained before.

We have also take into account that the `dequeue` operation is $O(1)$ **amortized**. If Outbound stack is empty it will transfer the elements from the Inbound stack, and this is of time complexity $O(n)$.



With the information you have received, open the Moodle environment and implement a Stack based Queue and the two important methods `enqueue` and `dequeue`.

Notice

Don't continue until you finish the exercise.

List based queue

We can also implement the queue with a Doubly Linked List as the insert and remove operations from/to either ends are of time complexity $O(1)$. Most of the functionality is already there. We can decide, for example, that our Queue enqueues from the front (head) of the list and dequeues from the end (tail) of the list.

So the `enqueue` method for this class could benefit from the `insert` method of the `DoublyLinkedList` class. We could also use the `remove` method, but it traverses the list to find the node to remove. So to remove the tail it would have to go through the whole list until the end. Hence it is of time complexity $O(n)$. Fortunately in the previous chapter, there was implemented a `pop` method for the Doubly Linked List, so `dequeue` can just call it.

```
class ListBasedQueue(DoublyLinkedList):
    def __init__(self):
        super().__init__()

    def __repr__(self):
        plural = '' if self._size == 1 else 's'
        values = ','.join([c for c in self])
        return f'<ListBasedQueue ({self._size} element{plural}): [{values}]'

    def enqueue(self, data):
        self.insert(0, data)

    def dequeue(self):
        return self.pop()
```

Node based Queue

After all we have seen, it should be quite easy to implement a Queue class from scratch that is based in simple nodes. It is basically a Double Linked List but stripped down to support just two actions: enqueue and dequeue.

So let's do just that. Your task now is to implement a custom class that defines a Queue and implements just two actions: enqueue and dequeue. It would also need to implement a `__repr__()` method so the exercise can be evaluated.

Open the Moodle environment and complete the exercise number 4.

> **Notice**
>
> Don't continue until you finish the exercise.

Finally we can take into use the Queue structure. You will write a function that accepts a list of numbers and make pairs with them. Every pair has to have an even number and an odd number. And the pairs should be formed in order: always the first available even number with the first available odd number. The function should return a list with the pairs as tuples.

To implement this function create two queues and traverse the number list. With every number, check if it is odd or even and check if there is a pair available in the appropriate queue. If not enqueue it.

Open the Moodle environment and complete the exercise number 5.

> **Notice**
>
> Don't continue until you finish the exercise.